GMRIT Deemed to be University

Empowering Minds, Shaping Future

LAB HANDOUT

Dated: 01.07.2026

Academic Year	: 2026-27
Degree	: B.Tech
School	: School of Advanced Computing
Program	: Computer Science and Engineering
Semester	: 3
Regulation	: AR23 (Autonomous – JNTUGV Regulation)
Course Title	: JAVA LAB
Course Code	: 23CS308
L-T-P-C Structure	: 0-0-1-1.5
Instructor(s)	: Mr. G. Dharma Raju, Mr. G Suneel, Dr. Radha Rani, Mr. G. Ravi Kumar Mrs. P Madhavi, Mr. G Surendar
Course coordinator	: Mr. G. Surendar

Course Description:

This course introduces the fundamental concepts of programming from an object-oriented perspective and the necessity to learn the importance of object-oriented programming (OOP) and the difference between the traditional procedure-oriented programming and object-oriented programming. In the structured oriented programming languages, methods define the structure of the programs, they are basic building blocks and data has secondary role while in object-oriented programming languages the data has the principle role. Methods belong to the data, without the data, the method does not have any meaning (Except static methods). Data and methods together make up the object. OOP tries to model the real world.

This course introduces the basic programming in java. Concepts and usage of class, object and constructors are introduced. Java as a class-based and pure OOP language is used to demonstrate and implement appropriate concepts and techniques. The students are exposed to the concepts, fundamental syntax, and the thought processes behind object-oriented programming. At the end of the course the students will be able to use the concepts of data abstraction, inheritance, polymorphism, exception handling and Threads. The course also introduces the concepts of developing Applet and event handling which enable them to design interactive applications.

Scope and Objective:

This course is designed to enable the students to:

- Learn and make use of SDK environment for creating and debugging programs using java
- Learn about classes and objects
- Understand the purpose of polymorphism, overloading and interfaces
- Implements application programs using different types of inheritance

- Create and import packages in developing projects using java
- Explore concepts of concurrent programming by using multithreading
- Handle runtime errors through exception handling mechanism
- Write interactive applications through applets

Course Outcomes:

At the end of the course the students will be able to:

1. Make use of JAVA SDK environment to create – debug and run java programs
2. Create applications based on code reusability
3. Develop programs using threads
4. Develop and debug real time problems using exception handling
5. Design Graphical User Interface using AWT components.
6. Design Graphical User Interface using Swing components.

Reference Manual/Books:

1. JAVA Lab manual, Department of CSE, CSE-AIML and CSE-AIDS, GMRIT, Rajam

CO-PO Mapping:

COs	PO1	PO2	PO3	PO4	PO10	PO12	PSO2
1	3	3	1	1	2	2	2
2	3	3	2	1	2	2	2
3	3	3	2	2	2	2	2
4	3	3	2	3	2	3	3
5	3	3	3	2	2	2	3
6	3	3	3	2	2	3	3

List of Experiments:

1. Case Study: Permutations and Combinations Calculator

   User Input:

   - The program should prompt the user to enter the total number of elements (n) and the number of elements to be selected (r).
   - Validate the input to ensure that both n and r are positive integers, and r is less than or equal to n.

   Functional Description:

   - Create a method **calculatePermutations** that takes two integers n and r as input and returns the number of permutations.
     Formula *nPr = n! / (n - r)!, where "!" denotes factorial*.

   - Create a method **calculateCombinations** that takes two integers n and r as input and returns the number of combinations.
     Formula *nCr = n! / (r! * (n - r)!).*

- Implement a separate method to calculate the factorial of a given integer.

Sample Output:

Enter the total number of elements (n): 5

Enter the number of elements to be selected (r): 3

Permutations (nPr): 60

Combinations (nCr): 10

2. Conversion Utility that allows users to convert measurements between different units.
User Input:

- The program should prompt the user to enter positive integer value for conversion type.
- The program should prompt the user to enter the positive input (feet/ meters/ miles/ kilometers) based on user option.

Functional Description:

- utilize method overloading to handle conversions between various units, such as feet to meters, miles to kilometers, and vice versa.
formula: feet to meters - feet * 0.3048
miles to kilometers - miles * 1.60934

Sample Output:

Measurement Conversion Utility

1. Feet to Meters

2. Meters to Feet

3. Miles to Kilometers

4. Kilometers to Miles

5. Exit

Enter your choice: 3

Enter the distance in miles: 5

5 miles is approximately 8.046 kilometers.

3. Case Study: **BankAccount** Class with Constructor Overloading
Functional Description:
   - Create a **BankAccount** class to represent a bank account. The class should have private attributes such as **accountNumber** (a unique identifier for the account), **accountHolderName**, and **balance** (representing the current account balance).
   - The class should have the following constructors:

- **BankAccount(String accountNumber, String accountHolderName)**: Create an account with a zero balance.
- **BankAccount(String accountNumber, String accountHolderName, double initialBalance)**: Create an account with the specified initial balance.

- Implement methods to:
  - **deposit(double amount):** Add the given amount to the account balance.
  - **withdraw(double amount):** Subtract the given amount from the account balance if sufficient funds are available.

Sample Output:

Welcome to the Bank Account System
Creating a new account...
Enter the account holder's name: John Doe
Enter the initial balance: 500.0
Account created successfully.
Account Number: ACCT123456
Account Holder: John Doe
Balance: $500.00
What would you like to do?
1. Deposit
2. Withdraw
3. Exit
Enter your choice: 1
Enter the amount to deposit: 250.0
$250.00 deposited successfully.
Updated Balance: $750.00

4. Implement inheritance concept for below case study:
   Note: class **FictionBook** extends class **LibraryBook**, class **ClassicBook** and class **ScienceFictionBook** extends from **FictionBook**

   - **LibraryBook**: Represents a general library book with attributes like title, author, and year, along with a method **displayBookInfo()** to display book details.
   - **FictionBook**: Extends **LibraryBook** and adds an additional attribute genre.
   - **ClassicBook**: Extends **FictionBook** and adds an additional attribute publisher.
   - **ScienceFictionBook**: Extends **FictionBook** and adds an additional attribute edition

5. Case Study: Abstract Class for Finding Areas of Different Shapes
   User Input:

   - The program should prompt the user to enter positive integer value to calculate type of area.
   - The program should prompt the user to enter the input based on user option.
     - **Circle**: Requires the radius as an input parameter
     - **Rectangle**: Requires the length and width as input parameters.
     - **Triangle**: Requires the base and height as input parameters.
     - **Square**: Requires the side length as an input parameter.

   Functional Description:

- Create an abstract class **Shape** that will serve as the base class for different shapes. Include an abstract method **calculateArea()**
- Create concrete subclasses that extend the **Shape** class for different shapes like **Circle**, **Rectangle**, **Triangle** and **Square**.

Sample Output:

Select a shape:

1. Circle

2. Rectangle

3. Triangle

4. Square

5. Exit

Enter your choice: 1

Enter the radius of the circle: 5

The area of the selected shape is: 78.53981633974483

6. Implement Dynamic Method Dispatch in Java.
7. i. Create an **ArrayList** of integers to store a collection of numbers. Implement methods to add elements to the list, remove elements, update elements, and search for elements in the list.
   ii. Create a Java program that accepts a string input from the user. Use the **StringTokenizer** class to tokenize the input string into separate tokens based on specified delimiters (e.g., space, comma, etc.).

8. Demonstrating Mutability Differences between **String**, **StringBuilder**, and **StringBuffer** in Java
   - Explain the Concept in 100 words.
   - Perform string manipulation operation concatenation on each class
   - Print the results of each class after modification operation
9. Demonstrate multiple inheritance-like behavior achieved through interface implementation.
10. Case Study: Java Program Illustrating Multiple Catch Clauses with Finally Block

Functional Description:

- Implement exception handling for possible exceptions that may occur during the division, such as **ArithmeticException** and **NumberFormatException**.
- Use multiple catch clauses to handle **ArithmeticException** and **NumberFormatException** separately.
- Use a **finally** block to execute code that should be performed regardless of whether an exception occurred or not

Sample Output:

Enter the dividend: 10

Enter the divisor: 0

Division by zero is not allowed.

Execution completed.

11. create your own custom exception classes to handle specific situations or errors that are not covered by the built-in exception classes.

Functional Description:

- Create a custom exception class that extends the **Exception** class (or any other appropriate superclass, depending on your requirements).
- Define the custom exception class to represent a specific error situation, such as **InvalidAgeException**, which is thrown when an invalid age is provided.
- Use the custom exception class in a method to throw the exception when a certain condition is met, such as when the age is less than zero or exceeds a specific limit.

Constraints:

The program should prompt the user to enter age integer value. Here, age<0 or age>120

Sample Output:

Enter age:-5

Exception: Invalid age. Age must be between 0 and 120.

12. creates three threads by extending the **Thread** class. Each thread will display a specific message at a regular interval. The messages to be displayed are as follows:
First thread: "**Good Morning**" every 1 second
Second thread: "**Hello**" every 2 seconds
Third thread: "**Welcome**" every 3 seconds
Note: implement the same by using **Runnable** interface.
13. Case Study: Java Program for Producer-Consumer Problem

User Input:

- The program should prompt the user to enter positive integer values for **bufferSize**
- The program should prompt the user to enter positive integer values to **produceCount** and **consumeCount**

Functional Description:

- Buffer Implementation:
  - Create a shared buffer with a limited size (e.g., an array or an ArrayList) to hold the produced items.
- Producer & Consumer Implementation:
  - Create a Producer class & Consumer class that implements the Runnable interface.
  - The Producer class should produce items and put them into the buffer.
  - The Consumer class should consume items from the buffer.
  - Implement synchronization to avoid race conditions when accessing the buffer.

Sample Output:

Enter Buffer Size:5

Enter Procedure Count:5

Enter Consumer Count:5

Produced: 0

Consumed: 0

Produced: 1

Consumed: 1

Produced: 2

Consumed: 2

Produced: 3

Consumed: 3

Produced: 4

Consumed: 4

14. Case Study: Java Program for a Basic Calculator using AWT
    In this case study, we'll implement a Java program to create a basic calculator using AWT. The calculator will have a simple graphical user interface (GUI) with buttons for numbers, arithmetic operations, and a display to show the result of calculations.
    AWT Implementation:
    - Create a Java AWT to implement the calculator GUI.
    - The AWT should have buttons for digits (0 to 9), arithmetic operations (addition, subtraction, multiplication, division), and an equal (=) button to perform calculations.

    Display and Input:
    - Implement a display area where the user can see the entered numbers and the result of calculations.
    - Provide a way for users to input numbers and arithmetic operations using the GUI buttons.

    Basic Calculations:
    - Implement methods to handle basic arithmetic calculations (addition, subtraction, multiplication, division).
    - Perform calculations based on the user's input and display the result in the GUI.

    User Interface:
    - Design a user-friendly GUI with appropriate labels, buttons, and display areas.
    - Use layout managers to organize the components effectively.
15. Case Study: Java Program for Temperature Converter using AWT
    In this case study, we'll implement a Java AWT to create a Temperature Converter. The AWT will allow users to convert temperatures between Celsius and Fahrenheit. The user will input a temperature value in one unit, and the AWT will display the converted temperature in the other unit.

AWT Implementation:

- Create a Java AWTpplet to implement the temperature converter GUI.
- The AWT should have input fields for the temperature value and buttons to select the source and target temperature units (Celsius or Fahrenheit).
- Use appropriate layout managers to organize the components effectively.

Conversion Logic:

- Implement methods to handle temperature conversions between Celsius and Fahrenheit.
- The temperature conversion formulas are:
  - Fahrenheit to Celsius: C = (F - 32) * 5 / 9
  - Celsius to Fahrenheit: F = (C * 9 / 5) + 32

User Interface:

- Design a user-friendly GUI with appropriate labels, input fields, and buttons.
- Display the converted temperature in the GUI after the conversion is performed.

16. Case Study: Java Swing Application for Job Application Form

In this case study, we'll implement a Java Swing application with a JFrame that contains fields for name, age, email, and qualifications. The application will validate the form data when the user clicks the "Submit" button. If all fields are filled, the data will be written to a file named "application_form.txt."

Swing Application:

- Create a Java Swing application with a JFrame to display the job application form.
- Design the form with labels for each field (name, age, email, qualifications) and corresponding input fields (text fields or other appropriate components).
- Add a "Submit" button to the form to allow users to submit their job application.

Data Validation:

- Implement validation for the form data to ensure that all fields are filled before submission.
- Display error messages for missing fields or invalid data.

Data Writing:

- Create a method to write the validated form data to a file named "application_form.txt."
- The file should be created in the current working directory.
- Use appropriate file handling techniques to write the data.

User Interaction:

- Allow users to input their name, age, email, and qualifications using the form fields.
- Trigger the form data validation when the user clicks the "Submit" button.
- Display appropriate messages to inform the user about the success or failure of form submission.

List of Augmented Experiments

- New Patient Registry Management System
- Restaurant Billing Management System
- Library Management System
- ATM Management System
- Bus Ticket Booking Management System
- Movie Ticket Booking Management System
- Queuing Management System
- Attendance Management System

- Medical Store Billing Management System
- Text Editor Projects in Java
- Google Search Engine Filter
- Electronic voting System
- Day Planner
- Library management System
- Personal Finance Management System

Course Plan:

Lab session No.	CO	Topics to be covered
1	CO1	Implement Class and Object concept for **Permutations and Combinations Calculator** case study
2	CO1	Conversion Utility that allows users to convert measurements between different units.
3	CO1, CO4	Implement Constructor Overloading concept for **BankAccount** case study
4	CO2	Implement inheritance concept for **LibraryBook** case study
5	CO2	Implement Abstract Class concept for **Finding Areas of Different Shapes** Case study
6	CO2	Implement Dynamic Method Dispatch in Java.
7	CO2, CO4	i. Create an **ArrayList** of integers to store a collection of numbers. Implement methods to add elements to the list, remove elements, update elements, and search for elements in the list. ii. Create a Java program that accepts a string input from the user. Use the **StringTokenizer** class to tokenize the input string into separate tokens based on specified delimiters (e.g., space, comma, etc.).
8	CO4	Demonstrating Mutability Differences between **String**, **StringBuilder**, and **StringBuffer** in Java • Explain the Concept in 100 words. • Perform string manipulation operation concatenation on each class • Print the results of each class after modification operation
9	CO2	Demonstrate multiple inheritance-like behavior achieved through interface implementation.
10	CO4	Implement Exception Handling concept for **Multiple Catch Clauses with Finally Block** case study
11	CO4	Create your own custom exception classes to handle specific situations or errors that are not covered by the built-in exception classes.
12	CO3	Implement the Thread creation by extending Thread class and by implementing Runnable interface

13	CO3	Implement synchronized threads concept for **Producer-Consumer Problem** case study
14	CO5	Implement GUI concept for **Basic Calculator** case study using AWT
15	CO5	Implement GUI concept for **Temperature Converting** case study using AWT
16	CO6	Implement GUI concept for **Job Application Form** case study using Swings

Evaluation scheme:

Component	Particular	Marks	Date & Time
Day to Day Performance & Augmented Experiment	No of Experiments completed and recorded	15 (5+5+5)	06-07-2026 to 02-11-2026
Internal Examination	180 minutes	15	02-11-2026 to 10-11-2026
External Examination	180 minutes	70	11-11-2026 to 16-11-2026
	Total	**100**	

Chamber Consultation Hour: Informed in the class in first week.
Venue: CSE Department
Notices: All Notices regarding the Course will be put in Notice Board.

Signature of the Course Instructors

Mr. G Dharma Raju - CSE

Mr. G Suneel – CSE

Dr. Radha Rani - CSE

Mr. G Ravi Kumar - CSE

Mrs. P. Madhavi – AI&ML

Signature of the Course-Coordinator

Mr. G Surendar-AI&DS

Signature of HoD